EECS E4750: Heterogeneous Computing
Assignment 2 Report

Kernel Fusion in PyCUDA & PyOpenCL

Fanghao Hu
UNI: fh2551

October 25, 2026

1 Experiments Setup

The experiments were run on a Google Cloud VM with an NVIDIA L4 GPU. Nsight Systems was version 2026.1.1 and Nsight Compute was version 2024.3.1.0 on VM and version 2026.1.0 on my computer. Although the assignment suggests a T4 reference, I was only able to create a L4 instance, so L4 is the bandwidth reference for following measurements.

2 Correctness

Both APIs passed all 11 test cases for all three implementations with `np.allclose(rtol=1e-5, atol=1e-6)`. The cases include one-element dimensions, non-multiples of 32, extreme aspect ratios, and isolated scale and shift. Figures 1 and 2 show the screenshots.

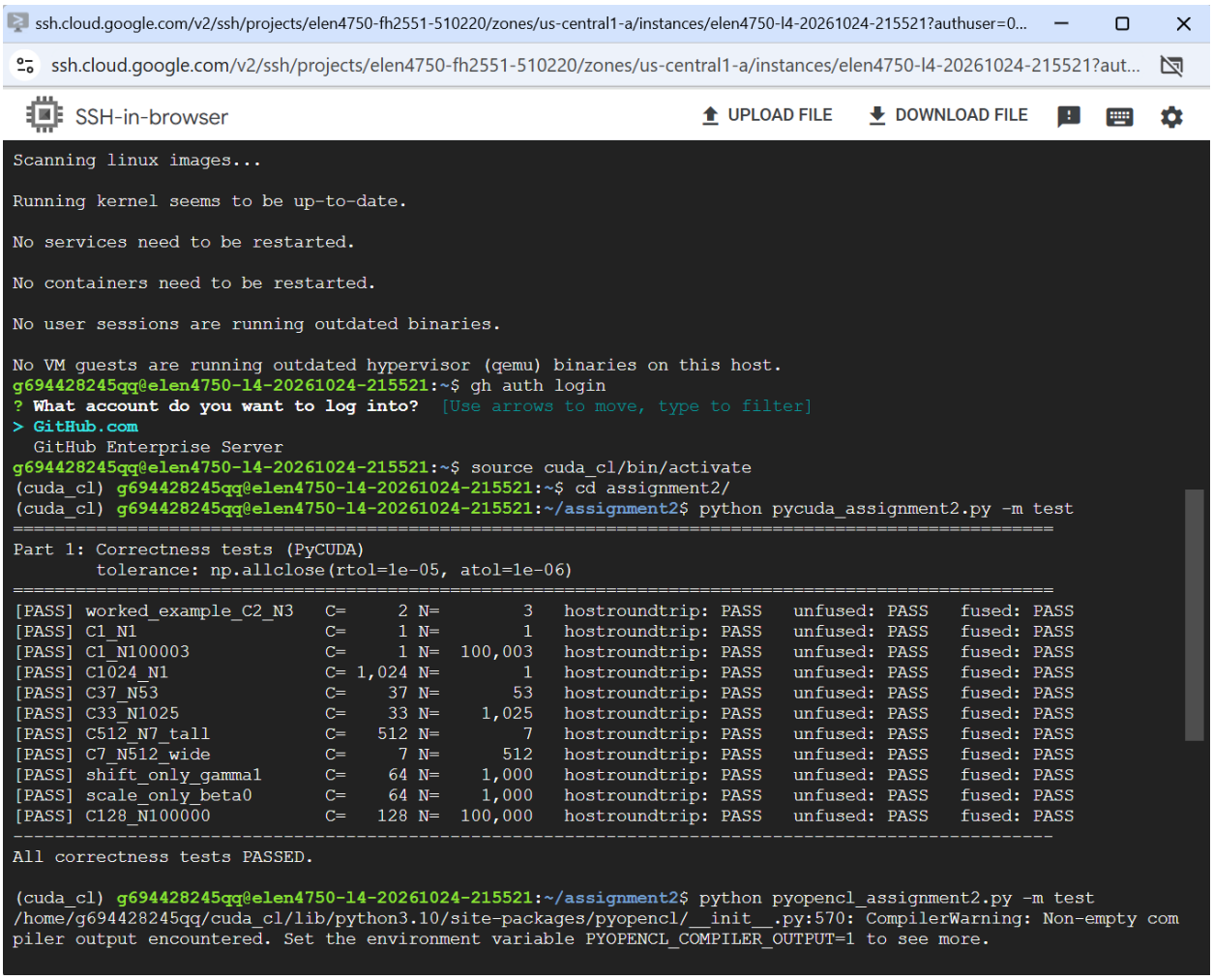


Figure 1: PyCUDA correctness tests

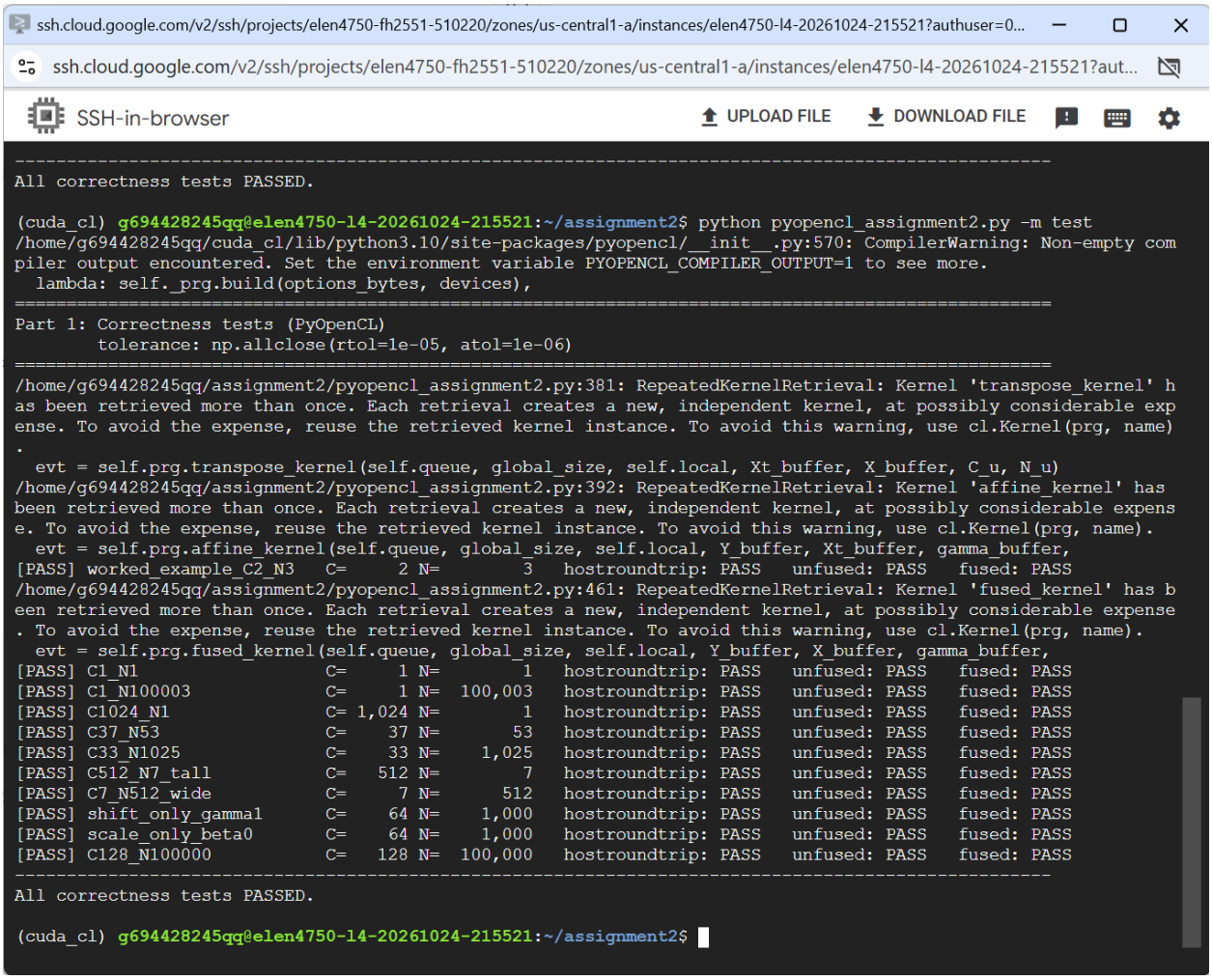


Figure 2: PyOpenCL correctness tests

3 Timing Results and Comparison

3.1 Size sweep at $C = 128$

Figures 3 and 4 plot kernel-only and end-to-end time for $N = 10$ through 10^6. In both APIs, the fused kernel is faster than the device-only two-kernel pair at the large sizes. At $N = 10^6$, the PyCUDA plot shows roughly 6–7 ms fused versus 10 ms unfused; the PyOpenCL plot shows a similar roughly 6 ms versus 10–11 ms separation. Removing one intermediate write, one intermediate read, and one launch lowers device work, but does not eliminate the transpose's strided output stores.

At small N, total time is much larger than kernel-only time, so fixed allocation, API, and transfer costs dominate. The host-round-trip total is highest because it adds a device-to-host and host-to-device copy of the full intermediate matrix. As N grows, those two extra copies scale with CN, so this version remains substantially slower end to end. The device-only unfused and fused totals are much closer because their host transfers are identical. The CPU NumPy curve is below the GPU total curves over most of the measured range; at the largest PyCUDA point, fused total time is slightly below NumPy, whereas the PyOpenCL total curves remain above it. Kernel-only GPU and end-to-end GPU times therefore answer different performance questions. Small PyCUDA

kernel-only points fluctuate, so a tiny-size speedup should not be over-interpreted.

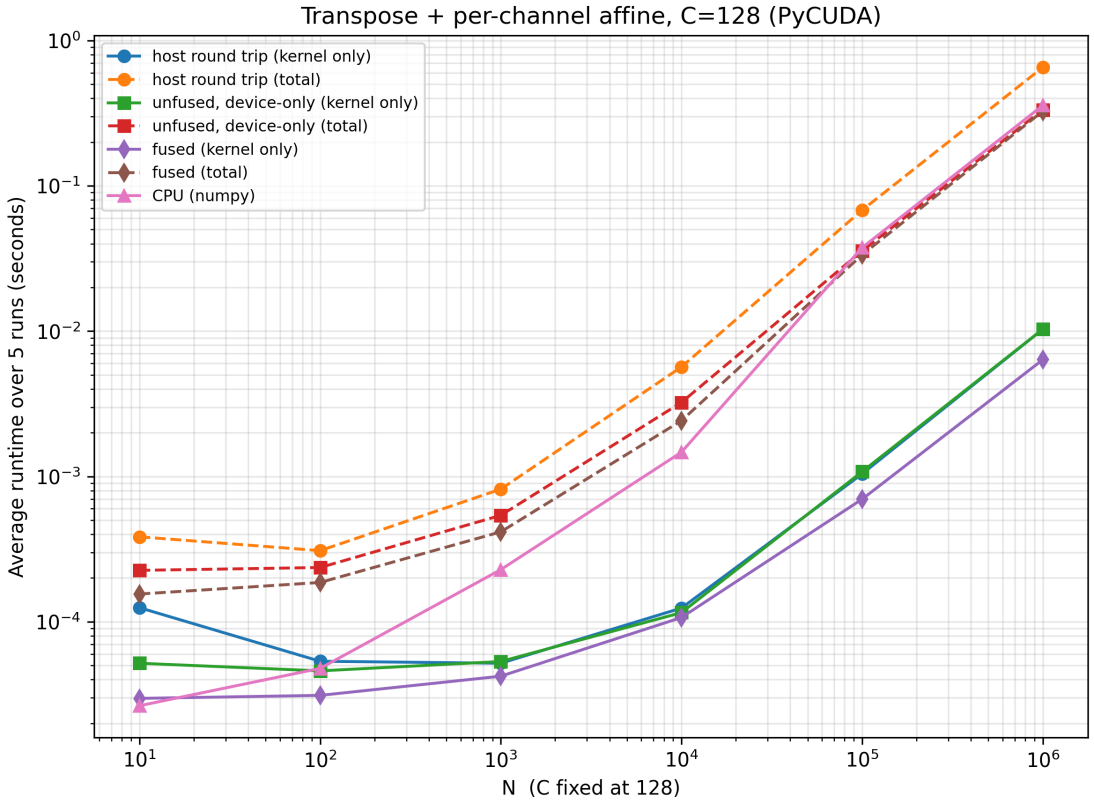


Figure 3: PyCUDA size sweep at $C = 128$

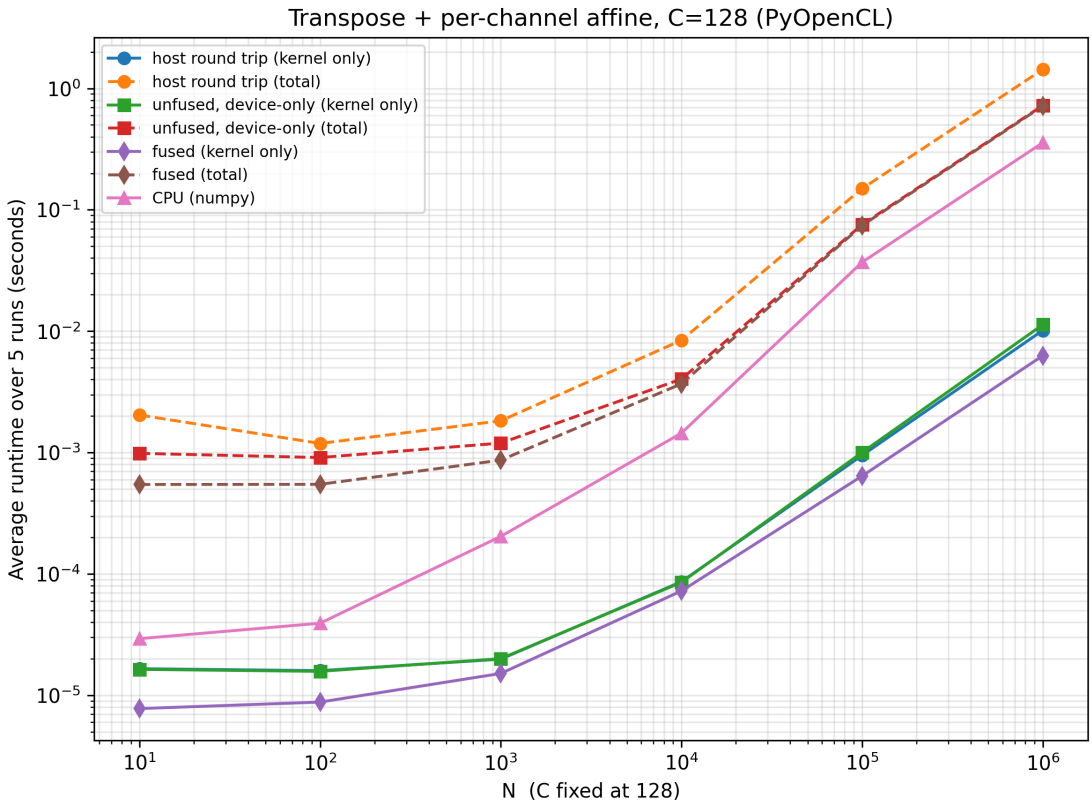


Figure 4: PyOpenCL size sweep at $C = 128$

3.2 Aspect-ratio sweep at fixed $CN = 2^{24}$

Figures 5 and 6 hold the element count at 16,777,216 while C increases from 32 to 32768. Every shape is divisible by 32, so work-item count and block utilization are fixed. The fused kernel-only curves are nearly flat, with a slight rise at large C: approximately 0.8–0.9 ms for PyCUDA and 0.75–0.85 ms for PyOpenCL. The two-kernel curves stay near 1.4–1.6 ms. This supports a memory traffic benefit from fusion that persists across the tested shapes. At the same time, total time changes little with aspect ratio because the same amount of input and output data is transferred. The mild rise of the fused curve at large C is consistent with shorter contiguous n segments, more channel boundaries, and changes in cache/transaction behavior.

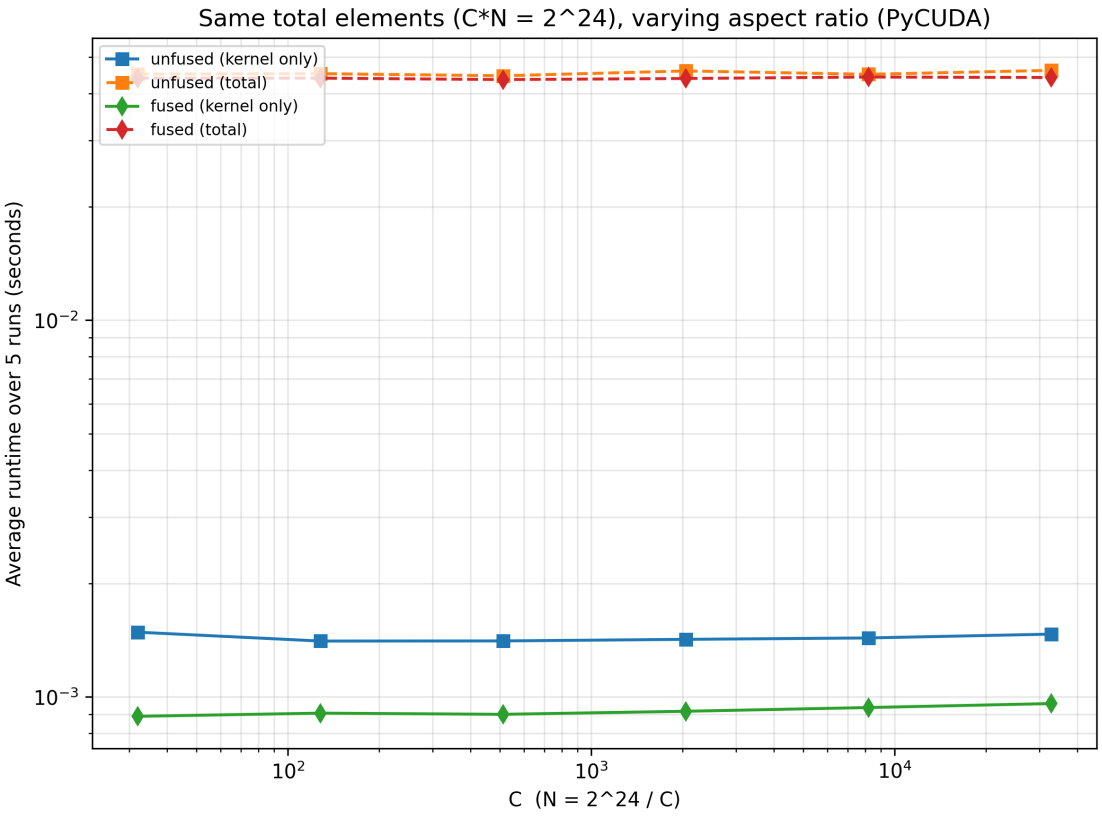


Figure 5: PyCUDA aspect-ratio sweep at fixed $CN = 2^{24}$

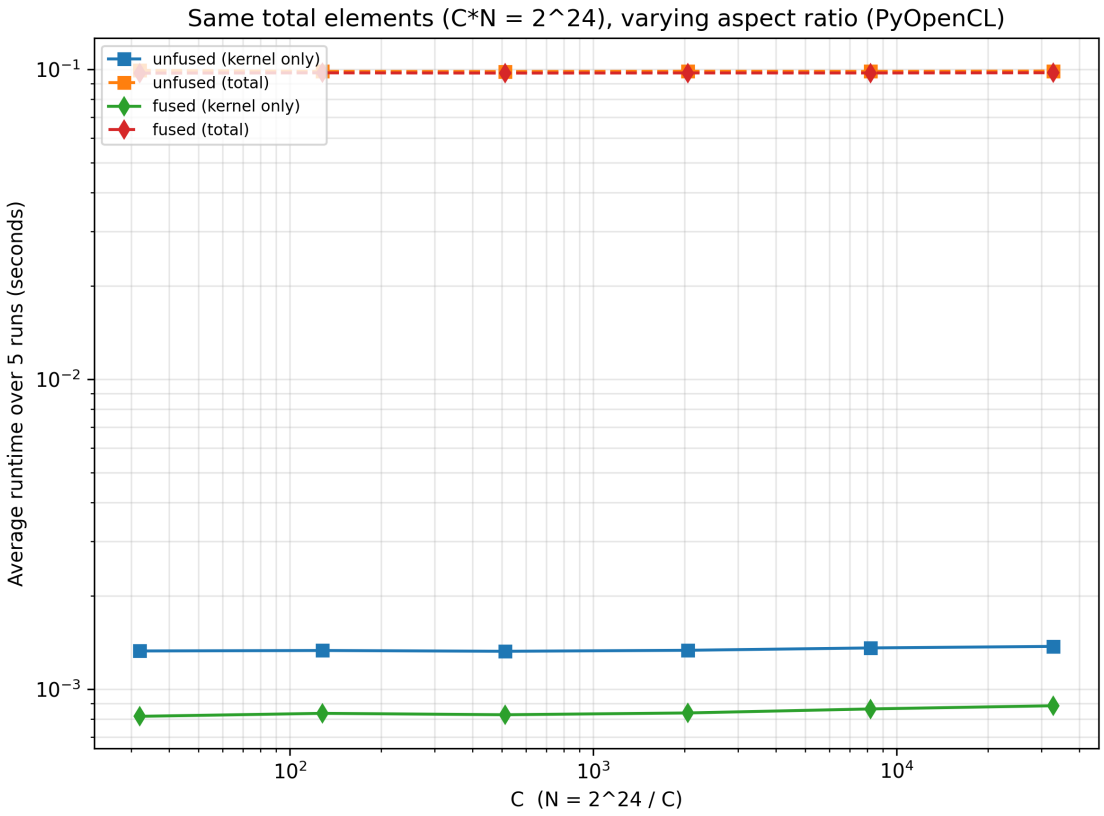


Figure 6: PyOpenCL aspect-ratio sweep at fixed $CN = 2^{24}$

4 Theory Questions

A1. Launch structure across the three versions

At $C = 256, N = 65536$, the host-round-trip and device-only unfused versions each launch two kernels; fused launches one. Figures 7, 8 and 9 show the required timelines. The device-only unfused sequence has an input HtoD copy, `transpose_kernel`, `affine_kernel`, and an output DtoH copy; there is no intermediate memcpy between its kernels. The zoomed view in Figure 10 shows a dispatch gap of 4.678 μs between the transpose ending and affine starting.

The per-version memcpy totals in Table 1 were summed from the same figure's `cuda_gpu_mem_time_sum` records. The extra round trip costs about 39.27 ms of GPU memcpy time relative to device-only unfused. Since the unfused and fused versions have the same host-transfer sequence, their remaining difference comes from the intermediate device buffer and second kernel launch.

Table 1: Memcpy duration at $C = 256, N = 65536$ from the Nsight Systems capture (ms; small γ and β copies are included)

Version	HtoD	DtoH	Total
Host round trip	27.960	53.580	81.540
Device-only unfused	15.017	27.253	42.269
Fused	14.024	27.085	41.109

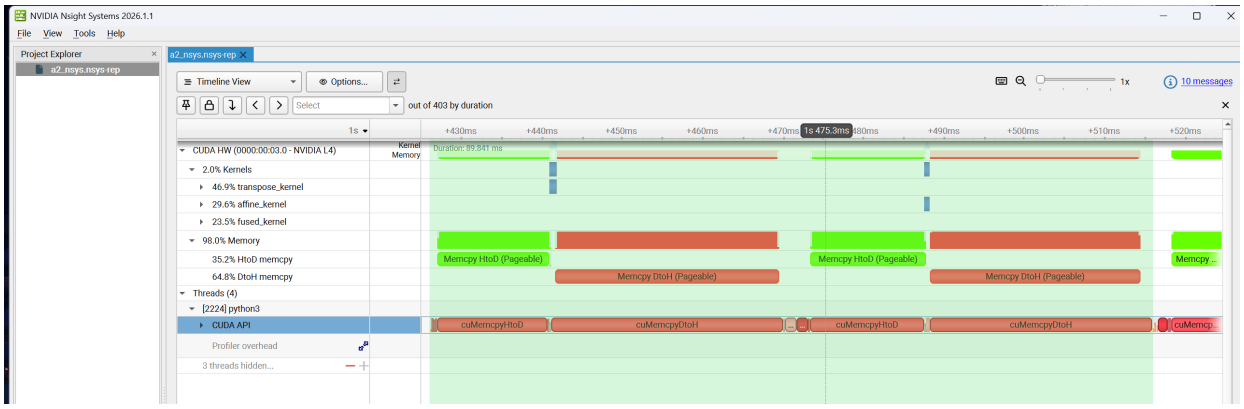


Figure 7: Nsight Systems timeline of ost-round-trip unfused at $C = 256, N = 65536$

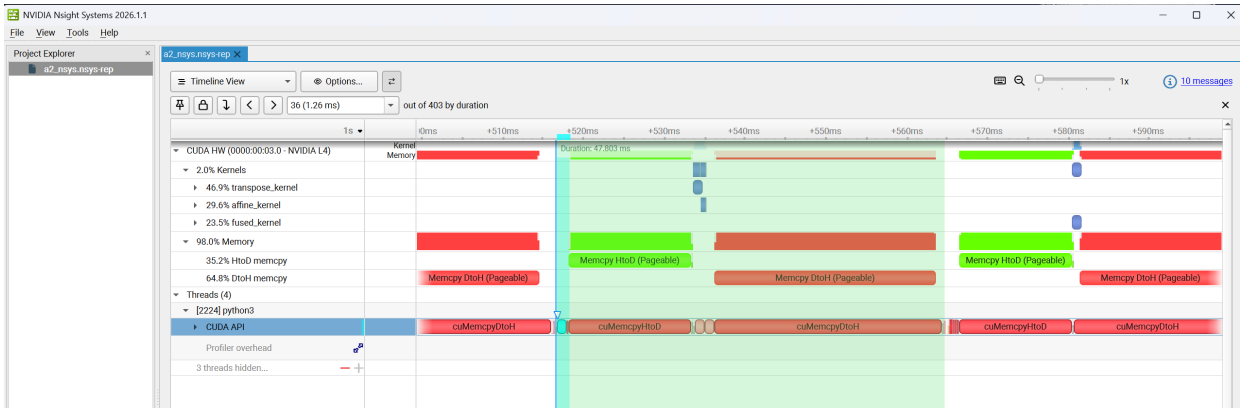


Figure 8: Nsight Systems timeline of device-only unfused at $C = 256, N = 65536$

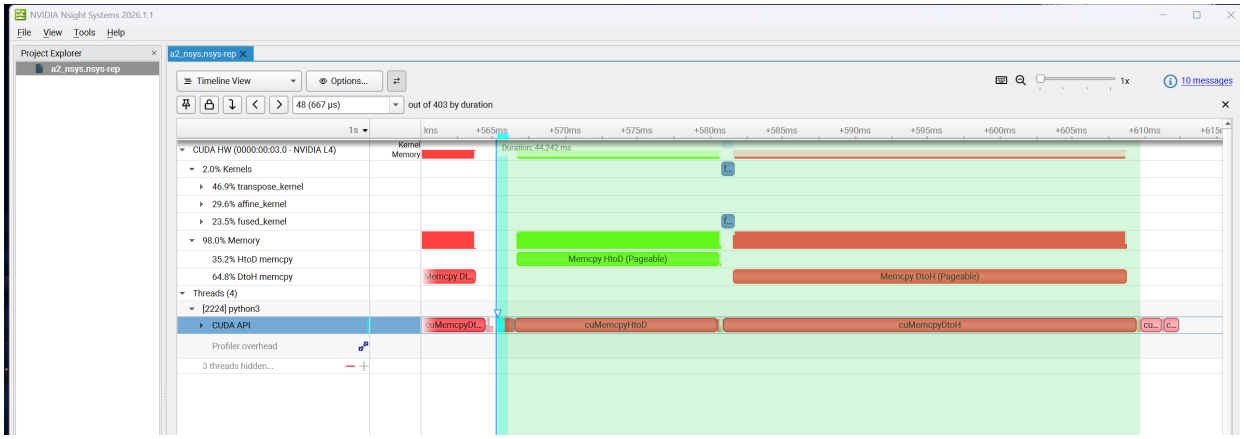


Figure 9: Nsight Systems timeline of fused at $C = 256, N = 65536$

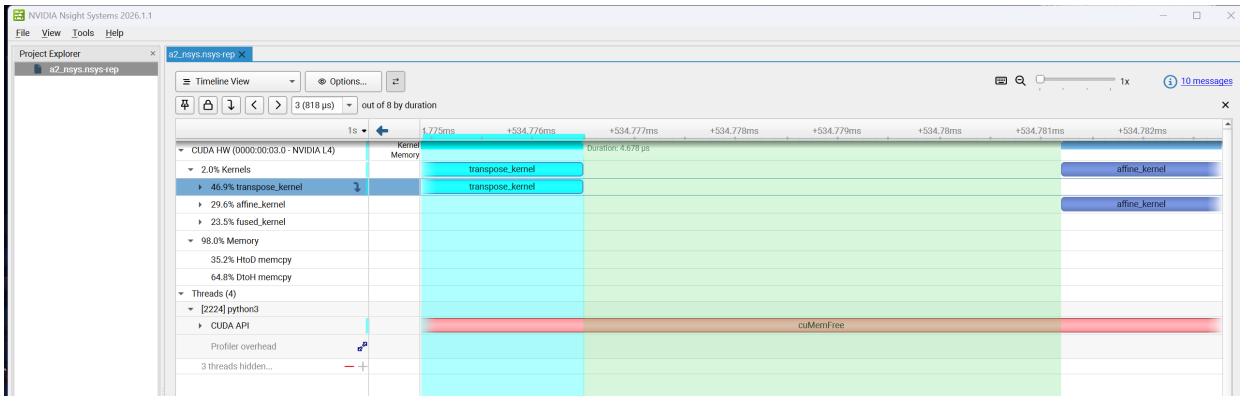


Figure 10: Zoomed unfused kernel gap: 4.678 μs

A2. Launch-overhead crossover

Table 2 shows the PyCUDA profiling ladder. The non-kernel column is $t_{\text{total}} - t_{\text{kernel}}$ and includes allocations and host transfers as well as dispatch, so it is not a pure launch-overhead measurement. At $N = 256$, the unfused and fused kernel times are 0.070 and 0.046 ms, versus non-kernel times of 0.552 and 0.331 ms. At $N = 1{,}048{,}576$, their kernel times rise to 21.462 and 13.132 ms while non-kernel time reaches 691.287 and 687.238 ms, mostly because full-array transfers grow with N. The approximately 4.7 μs dispatch gap measured at $N = 65536$ is below 1% of either kernel duration at that shape (about 0.8 and 0.5 ms in the Nsight Systems event trace). Thus by $N = 65536$ per-launch dispatch time is small relative to kernel work; the smaller $N = 4096$ rung is transitional. Fixed launch overhead makes fusion especially useful for small device-resident operations, while elimination of intermediate global-memory traffic keeps it useful for large ones. For calls that include host copies, transfer time can hide much of the end-to-end gain.

Table 2: Nsight Systems ladder; all values in ms, from the recorded profiling output

N	Version	Kernel	Total	Total − kernel
256	Unfused	0.070	0.622	0.552
256	Fused	0.046	0.378	0.331
4,096	Unfused	0.111	3.965	3.854
4,096	Fused	0.099	2.095	1.996
65,536	Unfused	1.423	47.812	46.389
65,536	Fused	0.917	44.263	43.335
1,048,576	Unfused	21.462	712.749	691.287
1,048,576	Fused	13.132	700.371	687.238

B1. Predicted versus measured DRAM bytes

At $C = 256, N = 65536$, one float32 matrix is $4CN = 67{,}108{,}864$ bytes (64 MiB). Ignoring the comparatively small coefficient arrays, the unfused pair should move four such matrices: X read, X_t written, X_t read, and Y written, or $16CN = 268.44$ million bytes (256 MiB). Fused should move two: X read and Y written, or $8CN = 134.22$ million bytes (128 MiB). Nsight Compute measured $80.63 + 57.29 + 74.14 + 52.53 = 264.59$ million bytes for the unfused pair, a measured/predicted ratio of 0.986; and $80.77 + 57.30 = 138.07$ million bytes for fused, a ratio of 1.029. Table 3 records the per-kernel measurements. Predictions count logical array access; DRAM counters observe physical

transactions after caching, sector granularity, writeback, and profiler effects. This can make the measured value slightly lower or higher than the simple byte count.

The fused kernel reached 78.39 GB/s of measured DRAM throughput, about 26.1% of the L4's 300 GB/s published peak[1]. The relatively low fraction is consistent with inefficient strided stores and a simple untiled transpose; it is not evidence that fewer logical bytes were moved.

Table 3: Nsight Compute metrics at $C = 256, N = 65536$. DRAM values are decimal Mbyte. Efficiencies are computed from measured bytes per 32-byte sector

Kernel	Duration (ms)	DRAM read	DRAM write	Store efficiency
Transpose	1.76	80.63	57.29	12.5%
Affine	0.51	74.14	52.53	100%
Fused	1.76	80.77	57.30	12.5%

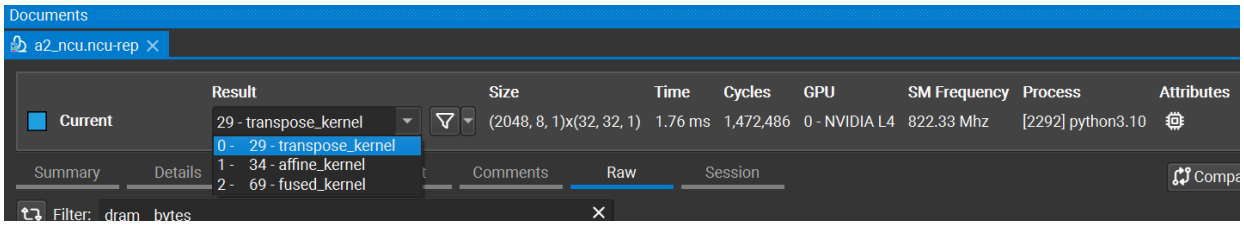


Figure 11: Nsight Compute IDs 0, 1, and 2 correspond to transpose, affine, and fused, respectively

ID	0	1	2
gpu__time_duration.sum [ms] (4.03 ms)	**1.76**	0.51	1.76

ID	0	1	2
dram__bytes_read.sum [Mbyte] (235.54 Mbyte)	**80.63**	74.14	80.77
dram__bytes_read.sum.pct_of_peak_sustained_elapsed [%]	**15.31**	48.76	15.30
dram__bytes_read.sum.per_second [Gbyte/s]	**45.90**	146.11	45.86
dram__bytes_write.sum [Mbyte] (167.12 Mbyte)	**57.29**	52.53	57.30
dram__bytes_write.sum.pct_of_peak_sustained_elapsed [%]	**10.88**	34.54	10.85
dram__bytes_write.sum.per_second [Gbyte/s]	**32.61**	103.52	32.53

Figure 12: Nsight Compute duration and DRAM-byte metrics for all three kernels

ID	0	1	2
dram__bytes.sum.per_second [Gbyte/s]	78.51	249.63	**78.39**

Figure 13: Achieved DRAM throughput; fused (ID 2) is 78.39 GB/s

[1]NVIDIA, *L4 Tensor Core GPU*, product specifications: https://www.nvidia.com/en-gb/data-center/l4/.

B2. Global store efficiency

The store-efficiency screenshot reports 4, 32, and 4 useful bytes per 32-byte sector for transpose, affine, and fused, respectively: 12.5%, 100%, and 12.5%. For fused, the global-load metric is $22.67/32 = 70.8\%$ in this capture. This aggregate is below the near-100% value suggested by the assignment; it includes all global loads, including coefficient loads, rather than isolating just the contiguous X loads.

For `fused_kernel` and `transpose_kernel`, consecutive lanes advance n with c fixed. The read $X[c, n]$ advances by 4 bytes, but the write $Y[n, c]$ advances by $4C = 1024$ bytes at the profiled $C = 256$. The resulting scattered writes explain their low store efficiency. In `affine_kernel`, consecutive lanes advance c and both the X_t read and Y write advance by 4 bytes; those stores coalesce well.

Improving transpose store efficiency would require a shared-memory tiled transpose: threads load contiguous X values into a tile, synchronize, then read the tile in transposed order so neighboring lanes write contiguous Y addresses.

Fusion removes the intermediate array and a launch, but does not change this direct transpose's strided output-store pattern.

ID	0	1	2
smsp__sass_average_data_bytes_per_sector_mem_global_op_st.max_rate [byte/sector]	32	32	32
smsp__sass_average_data_bytes_per_sector_mem_global_op_st.ratio [byte/sector]	4	32	4

ID	0	1	2
smsp__sass_average_data_bytes_per_sector_mem_global_op_ld.max_rate [byte/sector]	32	32	32
smsp__sass_average_data_bytes_per_sector_mem_global_op_ld.ratio [byte/sector]	32	32	22.67

Figure 14: Global store efficiency for all kernels and global load efficiency for fused (ID 2), reported as bytes per 32-byte sector

C1. Aspect-ratio sweep

The fused runtime is almost flat as C grows from 32 to 32768, with a small increase at the largest C values in both APIs. Since CN, thread count, and full-work-group occupancy are held constant, there is no change in total useful work or boundary-thread waste. The fixed 4-byte contiguous X read and $4C$-byte strided Y write within a warp explain why the kernel remains dominated by the same access pattern. Modest variation can come from changed row lengths and memory-system behavior.

C2. Kernel fusion in general

Kernel fusion combines successive operations into a single kernel so that an intermediate result can stay in registers or shared memory instead of being written to and reread from global memory, and fewer kernels must be launched. For example, fusing matrix multiplication with bias addition and ReLU can apply both operations while each output tile is still available inside the matrix-multiplication kernel, avoiding a separately materialized pre-activation matrix.